

Python Programming Fundamentals Cheat Sheet

Package/Method	Description	Syntax and Code Example
AND	Returns 'True' if both statement1 and statement2 are 'True'. Otherwise, returns 'False'.	Syntax: <pre>statement1 and statement2</pre> Example:<pre>marks = 88 percentage = 87 if marks >= 80 and percentage >= 80: print("qualify for honors") else: print("Not qualified for honors") # Output - qualify for honors</pre>
Class Definition	Defines a blueprint for creating objects and defining their attributes and behaviors.	Syntax: <pre>class ClassName: # Class attributes and methods</pre> Example: <pre>class Person: def __init__(self, name, age): self.name = name self.age = age</pre>
Define Function	A 'function' is a reusable block of code that performs a specific task or set of tasks when called.	Syntax: <pre>def function_name(parameters): # Function body</pre> Example: <pre>def greet(name): print("Hello, " + name)</pre>
Equal(==)	Checks if two values are equal.	Syntax: <pre>variable1 == variable2</pre> Example 1: <pre>5 == 5</pre> returns True Example 2: <pre>age = 25 age == 30</pre> returns False
For Loop	A 'for' loop repeatedly executes a block of code for a specified number of iterations or over a sequence of elements (list, range, string, etc.).	Syntax: <pre>for variable in sequence: # Code to repeat</pre> Example 1: <pre>for num in range(1, 10): print(num)</pre> Example 2: <pre>fruits = "apple", "banana", "orange", "grape", "kiwi" for fruit in fruits: print(fruit)</pre>
Function Call	A function call is the act of executing the code within the function using the provided arguments.	Syntax: <pre>function_name(arguments)</pre> Example: <pre>greet("Alice")</pre>
Greater Than or Equal To(>=)	Checks if the value of variable1 is greater than or equal to variable2.	Syntax: <pre>variable1 >= variable2</pre> Example 1: <pre>5 >= 5 and 9 >= 5</pre> returns True Example 2: <pre>quantity = 100 minimum = 80 quantity >= minimum</pre> returns True
Greater Than(>)	Checks if the value of variable1 is greater than variable2.	Syntax: <pre>variable1 > variable2</pre> Example 1: <pre>5 > 5 and 9 > 6</pre> returns True Example 2: <pre>age = 20 max_age = 25 age > max_age</pre> returns False
If Statement	Executes code block 'if' the condition is 'True'.	Syntax: <pre>if condition: # Code to execute if condition is True</pre> Example: <pre>if temperature > 30: print("It's a hot day!")</pre>
If Else Else	Executes the first code block if condition1 is 'True', otherwise checks condition2, and so on. If no condition is 'True', the else block is executed.	Syntax: <pre>if condition1: # Code to execute if condition1 is True elif condition2: # Code to execute if condition2 is True else: # Code to execute if no condition is True</pre> Example: <pre>score = 85 if score >= 90: print("You are an A+") elif score >= 80: print("You are a B+") elif score >= 70: print("You are a C+") else: print("You need to work harder!")</pre>
If Else Statement	Executes the first code block if the condition is 'True', otherwise the second block.	Syntax: <pre>if condition: # Code to execute if condition is True else: # Code to execute if condition is False</pre> Example: <pre>if age >= 18: print("You're an adult.") else: print("You're not an adult yet.")</pre>
Less Than or Equal To(<=)	Checks if the value of variable1 is less than or equal to variable2.	Syntax: <pre>variable1 <= variable2</pre> Example 1: <pre>5 <= 5 and 3 <= 5</pre> returns True Example 2: <pre>size = 30 max_size = 40 size <= max_size</pre> returns True
Less Than(<)	Checks if the value of variable1 is less than variable2.	Syntax: <pre>variable1 < variable2</pre> Example 1: <pre>4 < 4</pre> returns True Example 2: <pre>score = 80 passing_score = 85 score < passing_score</pre> returns True
Loop Controls	'break' exits the loop prematurely 'continue' skips the rest of the current iteration and moves to the next iteration.	Syntax: <pre>for: # Code to repeat if condition: break else: # Code to execute after the loop ends</pre> Example 1: <pre>for num in range(1, 6): if num == 3: break print(num)</pre> Example 2: <pre>for num in range(1, 6): if num == 3: continue print(num)</pre>
NOT	Returns 'True' if variable is 'False', and vice versa.	Syntax: <pre>not variable</pre> Example: <pre>is_locked = False not is_locked</pre> returns True (if the variable is False (i.e., unlocked))
Not Equal(!=)	Checks if two values are not equal.	Syntax: <pre>variable1 != variable2</pre> Example: <pre>5 != 10 5 != 5</pre> returns True Example 2: <pre>countdown = 0 countdown != 0</pre> returns False
Object Creation	Creates an instance of a class (object) using the class constructor.	Syntax: <pre>object_name = ClassName(arguments)</pre> Example: <pre>person = Person("Alice", 25)</pre>
OR	Returns 'True' if either statement1 or statement2 (or both) are 'True'. Otherwise, returns 'False'.	Syntax: <pre>statement1 or statement2</pre> Example: <pre>"Obama" in ["Barack", "Michelle"] "Obama" in ["Barack", "Michelle"] or "Michelle" in ["Barack", "Michelle"]</pre> returns True
range()	Generates a sequence of numbers within a specified range.	Syntax: <pre>range(start, stop, step)</pre> Example: <pre>range(1, 10) # Generates numbers from 1 to 9 range(1, 10, 2) # Generates odd numbers from 1 to 9</pre>
Return Statement	'Returns' is a keyword used to send a value back from a function to its caller.	Syntax: <pre>return value</pre> Example: <pre>def add(a, b): return a + b</pre> result = add(5, 3)
Try-Except Block	Tries to execute the code in the try block. If an exception of the specified type occurs, the code in the except block is executed.	Syntax: <pre>try: # Code to try except ExceptionType: # Code to handle the exception</pre> Example: <pre>try: num = int(input("Enter a number: ")) except ValueError: print("Invalid input. Please enter a valid number.")</pre>
Try-Except-Else Block	Code in the 'else' block is executed if no exception occurs in the try block.	Syntax: <pre>try: # Code to try except ExceptionType: # Code to handle the exception else: # Code to execute if no exception occurs</pre> Example: <pre>try: num = int(input("Enter a number: ")) except ValueError: print("Invalid input. Please enter a valid number.") else: print("Valid input. Proceeding with the program.")</pre>

		<pre>if __name__ == '__main__': print("You entered:", name)</pre>
Try Except with Finally Block	Code in the 'finally' block always executes, regardless of whether an exception occurred.	Syntax: <pre>try: # Code that might raise an exception except <i>exception_type</i>: # Code to handle the exception finally: # Code that always executes</pre> Example: <pre>try: file = open("data.txt", "r") data = file.read() except FileNotFoundError: print("File not found.") finally: file.close()</pre>
While Loop	A 'while' loop repeatedly executes a block of code as long as a specified condition remains 'True'.	Syntax: <pre>while condition: # Code to repeat</pre> Example: <pre>count = 0 while count < 5: print(count) count += 1</pre>